DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2

SCALABILITY AUDIT OF SHOPSPHERE

AN ONLINE SHOPPING PLATFORM

Submitted by**: MADHUSUDHAN.D**

Registration Number:**192524214**

1. Problem Statement and Assumptions

ShopSphere currently has n = 100,000 products and expects the catalogue to reach n = 1,000,000. The audit considers five back-end modules and a separate nightly report generator. The assumed server rate is 10^8 basic operations per second, and the nightly processing window is 10 minutes (600 seconds).

One comparison, audit, or unit of recurrence work is treated as one basic operation. Runtime estimates divide the operation count by 10^8. Real-world performance can differ due to I/O, database latency, caching, memory, concurrency and constant factors.

(a) Understand the Problem

Module	**Input/basic operation**	**Growth and justification**
M1 Duplicate detector	n products; compare title and seller for every pair in nested loops.	$\Theta(n^2)$: each unordered pair is checked.
M2 API rate limiter	n requests; Audit(Request[j]) in inner loop; i doubles.	$\Theta(n)$: inner-loop sizes form 1 + 2 + 4 + ….
M3 S1 sequential search	n entries; compare target with each visited product.	Best $\Theta(1)$; average/worst $\Theta(n)$.
M3 S2 binary search	n sorted entries; compare with middle entry and halve interval.	$\Theta(\log n)$.

M4 current	n data items; four recursive calls of size n/2 plus linear merge.	$\Theta(n^2)$.
M4 redesign	n data items; two recursive calls of size n/2 plus linear merge.	$\Theta(n \log n)$.
M5 sales analytics	n records; n log n work at each recursive level.	$\Theta(n \log^2 n)$.

The likely bottlenecks are the quadratic M1, current M4, and report generator. M2 is linear because its loop counts form a geometric series. S2 is logarithmic because it discards half the search interval per comparison.

2. (b) M1 – Exact Comparison Count

For a fixed outer-loop value i, j runs from i + 1 to n − 1. Therefore, the inner loop performs n − i − 1 comparisons.

$$\mathbf{C_1(n) = \Sigma_{i=0}^{n-2} (n - i - 1)}$$

$$\mathbf{= (n - 1) + (n - 2) + \ldots + 2 + 1}$$

$$\mathbf{= 1 + 2 + \ldots + (n - 1)}$$

$$\mathbf{= n(n - 1)/2 = (n^2 - n)/2}$$

The leading term is $n^2/2$, so the time complexity is $\Theta(n^2)$. The loops use only counters i and j, so auxiliary space is $\Theta(1)$, excluding separate storage of duplicate results.

At n = 1,000,000: $C_1(n) = 1{,}000{,}000 \times 999{,}999 / 2 = 499{,}999{,}500{,}000$ comparisons. Time = $499{,}999{,}500{,}000 / 10^8 = 4{,}999.995$ seconds ≈ 83.33 minutes. M1 fails the 10-minute window.

3. (b) M2 – Exact Audit Count

The included i values are 1, 2, 4, …, 2^k, where $k = \lfloor \log_2(n - 1) \rfloor$. The inner loop performs i audits for each value.

$$\mathbf{C_2(n) = 1 + 2 + 4 + \ldots + 2^k}$$

$$\mathbf{= (2^{k+1} - 1)/(2 - 1) = 2^{k+1} - 1}$$

$$C_2(n) = 2^{(\lfloor \log_2(n-1) \rfloor + 1)} - 1$$

The $\Theta(n \log n)$ claim is incorrect. The outer loop runs $\Theta(\log n)$ times, but the inner loop is not n operations on every iteration. Summing the geometric series gives the tight bound $\Theta(n)$.

For $n = 1{,}024$, the included values are 1, 2, 4, …, 512. Thus $C_2(1{,}024) = 1 + 2 + \ldots + 512 = 2^{10} - 1 = 1{,}023$ audits.

4. (c) S2 – Backward Substitution

Binary search uses $T(n) = T(n/2) + 1$, with $T(1) = 1$. Each step performs one comparison and recurses on half the remaining entries.

$$\mathbf{T(n) = T(n/2) + 1}$$

$$\mathbf{= [T(n/4) + 1] + 1 = T(n/4) + 2 =}$$

$$\mathbf{[T(n/8) + 1] + 2 = T(n/8) + 3}$$

$$\mathbf{\dots}$$

$$\mathbf{= T(n/2^k) + k}$$

After k substitutions, the problem size is $n/2^k$. Stop when the size becomes 1:

$$\mathbf{n/2^k = 1 \ \Rightarrow \ n = 2^k \ \Rightarrow \ k = \log_2 n}$$

$$\mathbf{T(n) = T(1) + k = 1 + \log_2 n}$$

For n a power of two, this is the exact closed form under the given base case. For arbitrary n, the worst-case comparison count is about $\lfloor \log_2 n \rfloor + 1$, depending on the interval convention. Therefore time complexity is $\Theta(\log n)$.

Iterative and Recursive Space Complexity

Implementation	Time	Auxiliary space	Reason
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$	Uses a loop and a fixed number of variables.
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$	Call stack has one frame per halving level.

At n = 1,000,000, $\log_2(n) \approx 19.93$, so about 20 comparisons are needed. The iterative version has constant auxiliary space; the recursive version has logarithmic stack space. Index sorting or maintenance cost should be accounted for separately.

5. (d) M4 – Master Theorem

Current design: $T(n) = 4T(n/2) + n$

Quantity	Value
a	4
b	2

f(n)	n
n^(log_b a)	$n^{(\log_2 4)} = n^2$
Comparison	$f(n) = n = O(n^{(2-\varepsilon)})$, $\varepsilon = 1$
Case	Case 1: f(n) polynomially smaller
Result	$T(n) = \Theta(n^2)$

The recursive work dominates the linear merge work. When n doubles, the dominant running time grows approximately fourfold. Replicated partitions therefore create a scalability risk. **Proposed redesign:**
$T(n) = 2T(n/2) + n$

Quantity	**Value**
a	2
b	2
f(n)	n
n^(log_b a)	$n^{(\log_2 2)} = n$
Comparison	$f(n) = \Theta(n^{(\log_b a)})$
Case	Case 2: equal growth
Result	$T(n) = \Theta(n \log n)$

When n doubles, the redesigned cost ratio is approximately:

$$\mathbf{T(2n)/T(n) \approx [2n \log_2(2n)]/[n \log_2 n]}$$

$$\mathbf{= 2(\log_2 n + 1)/\log_2 n}$$

This ratio is slightly greater than two and approaches two as n grows. Remove replicated recursive work if it is not essential to correctness or fault tolerance, then validate output quality and consistency.

6. (d) M5 – Sales Analytics Engine

The recurrence is $T(n) = 2T(n/2) + n \log n$. Here a = 2, b = 2, and n^(log_b a) = n. The non-recursive work is $f(n) = n \log n$.

The simplest three-case Master Theorem presentation does not directly cover this extra logarithmic factor. The extended Case 2 applies because $f(n) = \Theta(n \log^1 n)$. A recursion tree provides the derivation.

Recursion-tree derivation

At level j, there are 2^j subproblems of size $n/2^j$. Each performs work proportional to $(n/2^j)\log_2(n/2^j)$. Total work at level j is:

$$\mathbf{2^j \times (n/2^j)\log_2(n/2^j) = n(\log_2 n - j)}$$

Let $n = 2^k$, so $k = \log_2 n$. Sum the internal levels j = 0 to k − 1:

$$\mathbf{\Sigma_{j=0}^{k-1} n(k - j) = n(1 + 2 + \ldots + k)}$$

$$\mathbf{= n\,k(k + 1)/2 = \Theta(n \log^2 n)}$$

The leaves contribute Θ(n), which does not change the dominant term. Therefore M5 has time complexity $\Theta(n \log^2 n)$. This is slower than Θ(n log n), but better than quadratic growth.

Meaning for ShopSphere

M5 is estimated to fit the 10-minute window under the given operation-rate model. Real execution should still be benchmarked because sorting, database access, and data movement may add cost.

7. (e) Best, Average and Worst Cases

Version	Best case	Average case	Worst case
S1 sequential	1 comparison: target is first.	If present uniformly: (n + 1)/2; if absent: n.	n comparisons: target last or absent.
S2 binary	1 comparison: target is middle entry.	Θ(log n); exact average depends on key distribution/tree shape.	About $\lfloor \log_2 n \rfloor + 1$ comparisons.

Expected S1 comparisons with probability p

If the product exists, assume each of the n positions is equally likely. The conditional expected number of comparisons is:

$$\mathbf{E[C \mid present] = (1 + 2 + \ldots + n)/n}$$

$$\mathbf{= [n(n + 1)/2]/n = (n + 1)/2}$$

If the product is absent, every record is checked, so E[C | absent] = n. By total expectation:

$$\mathbf{E[C] = p \times (n + 1)/2 + (1 - p) \times n}$$

Evaluation for n = 1,000,000 and p = 0.5

$$\mathbf{E[C] = 0.5 \times (1{,}000{,}001/2) + 0.5 \times 1{,}000{,}000}$$

$$= 250{,}000.25 + 500{,}000 = 750{,}000.25 \text{ comparisons}$$

The expected cost is about 750,000 comparisons per sequential search, which is $\Theta(n)$. S1 is unsuitable as the primary search method for a large catalogue with frequent lookups.

8. (f) Formal Proof: $f(n) = \Theta(n^2)$

The report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations. By definition, $f(n) = \Theta(n^2)$ if positive constants c_1, c_2 and n_0 exist such that $0 \le c_1 n^2 \le f(n) \le c_2 n^2$ for every $n \ge n_0$.

Lower bound

For every integer $n \ge 1$, $\log_2 n \ge 0$, so the remaining terms are non-negative:

$$\mathbf{f(n) = 3n^2 + 10n \log_2 n + 50 \ge 3n^2}$$

Choose $c_1 = 3$.

Upper bound

For integer $n \ge 1$, $\log_2 n \le n$, so $n \log_2 n \le n^2$. Also, $50 \le 50n^2$. Therefore:

$$\mathbf{f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2}$$

Choose $c_2 = 63$. Both bounds hold for all $n \ge 1$, so choose $n_0 = 1$:

$$\mathbf{3n^2 \le f(n) \le 63n^2, \text{ for all } n \ge 1}$$

Therefore $f(n) = \Theta(n^2)$, with $c_1 = 3$, $c_2 = 63$ and $n_0 = 1$.

Show f(n) is not O(n)

Suppose $f(n) = O(n)$. Then constants $c > 0$ and n_0 would exist such that $f(n) \le cn$ for all $n \ge n_0$. But $f(n) \ge 3n^2$, so $f(n)/n \ge 3n$, which grows without bound. Choose $n > c/3$ and $n \ge n_0$. Then $f(n) \ge 3n^2 > cn$, a contradiction. Hence f(n) is not O(n).

9. (g) Operation Counts and Running-Time Estimates

At $n = 1{,}000{,}000$, estimated seconds = operation count $\div 10^8$. The nightly deadline is 600 seconds.

Module	Operations (approx.)	Time estimate	Deadline
M1	499,999,500,000	4,999.99 s (83.33 min)	FAIL
M2	1,048,575	0.010486 s	PASS
S2	about 20 comparisons	0.000000 s	PASS

M4 current	about 2×10^{12}	19,999.99 s (333.33 min)	FAIL
M4 redesign	20,931,569	0.209316 s	PASS
M5	209,599,497	2.10 s (0.03 min)	PASS (estimate)
Report generator	3,000,199,315,736	30,001.99 s (500.03 min)	FAIL

M1: 499,999,500,000 / 10^8 = 4,999.995 seconds, approximately 83.33 minutes.

M2: $2^{20} - 1$ = 1,048,575 audits, approximately 0.0105 seconds.

S2: about 20 comparisons; this excludes the cost of building or updating the sorted index.

M4 current: using a unit-cost recurrence estimate, about $2n^2 - n \approx 2 \times 10^{12}$ operations, or 20,000 seconds (about 333 minutes).

M4 redesign: $n \log_2 n + n \approx 20.93$ million operations, or about 0.21 seconds.

M5: recursion-tree estimate ≈ 199 million operations, or about 2 seconds. Constants depend on the implementation.

Report generator: $f(n) \approx 3.0000002 \times 10^{12}$ operations, about 30,000 seconds = 500 minutes = 8 hours 20 minutes.

10. Redesign Recommendations and Conclusion

Priority	**Module**	**Recommendation**
Critical	M1 duplicate detector	Replace pairwise comparisons with hashing/grouping by (seller, normalized title), expected $\Theta(n)$, or sort by the composite key and compare adjacent entries, $\Theta(n \log n)$.

Critical	M4 current design	Remove unnecessary recursive replication and adopt the twobranch design, reducing $\Theta(n^2)$ to $\Theta(n \log n)$. Validate correctness and recommendation quality.
Critical	Nightly report generator	Eliminate the quadratic term using indexed aggregation, incremental summaries, grouped queries, or warehouse/OLAP methods.
Maintain	M2 rate limiter	Keep the doubling-window approach; its tight bound is $\Theta(n)$, not $\Theta(n \log n)$.
Maintain	S2 search	Use binary search over a maintained sorted index. Account separately for index preparation and update costs.
Monitor	M5 sales analytics	Estimated to fit the window; benchmark with realistic workloads and optimize repeated sorting if necessary.

Conclusion

The audit shows that quadratic algorithms are the main scalability risks. M1, the current M4 design, and the nightly report generator exceed the 10-minute window under the given model and must be redesigned. M2, S2, redesigned M4, and M5 are estimated to fit.

M1: $\Theta(n^2)$ | M2: $\Theta(n)$ | S1: $\Theta(n)$ | S2: $\Theta(\log n)$

M4 current: $\Theta(n^2)$ | M4 redesign: $\Theta(n \log n)$ | M5: $\Theta(n \log^2 n)$

Before submission, fill in the registration number and confirm the final page count in Microsoft Word.